UNIFIED VIBE CODING TUTORIAL WITH APP PROMPTS

"If a task, or even a question, comes up five times a month, it's a candidate for automation" – Azeem Azhar, founder of Exponential View

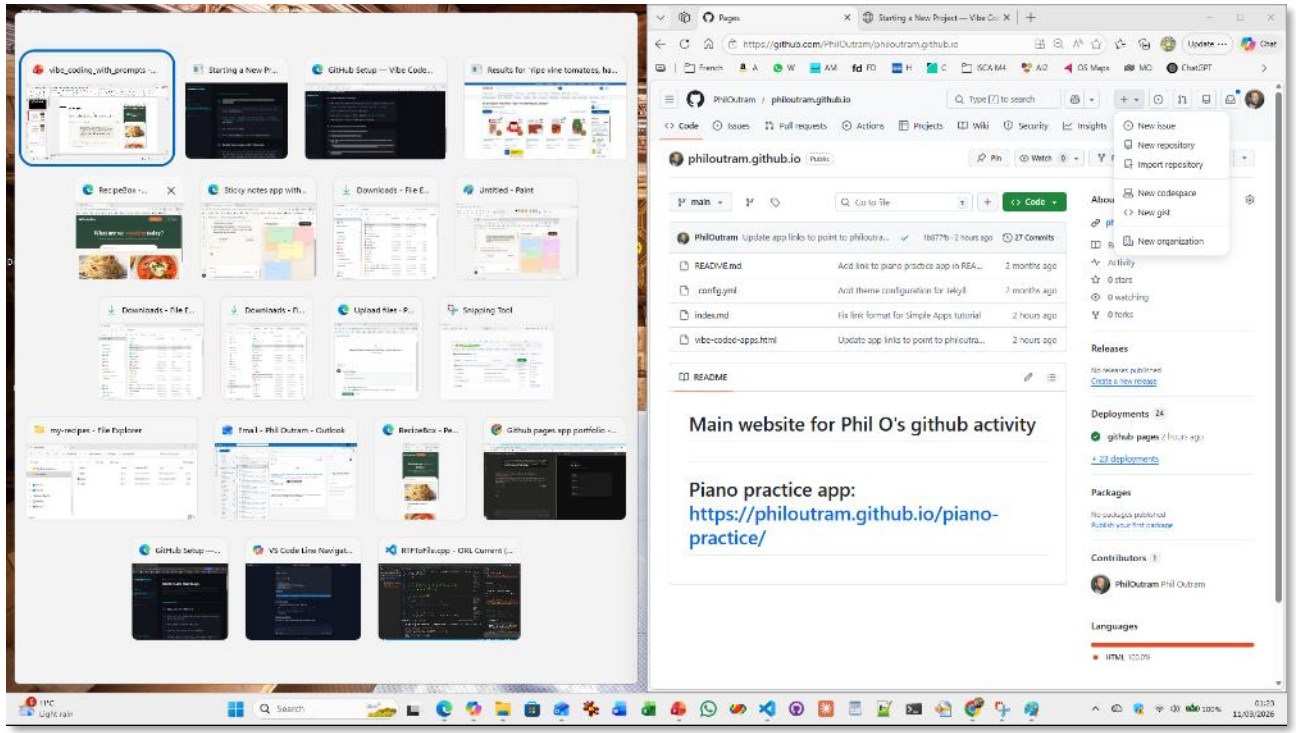
Comprehensive guide to coding and app-based prompts

HANDY HINT – SPLITTING YOUR SCREEN

To view the tutorial video and do the activities in your web browser at the same time, try splitting your screen half-and-half between your video and your web browser.

To do this…

1. Ensure both your **web browser** (Edge, Chrome, etc.) and **video app** (Teams, Zoom, etc.) are open.
2. Make sure your **video app** has the focus
3. Hold down the ⊞ windows key on your keyboard and press the -> right arrow key.
4. This moves the video app to fill the right half of your screen. At the same time, small thumb nails of your other windows are shown in the left half of the screen – this is for you to choose one to occupy the other half of the screen.
5. Click on your browser to fill that half of the screen.



THE SIMPLEST APPROACH

I started my vibe-coding by signing up for a dedicated vibe-coding tool, that lets you build apps with natural language (English) instructions.

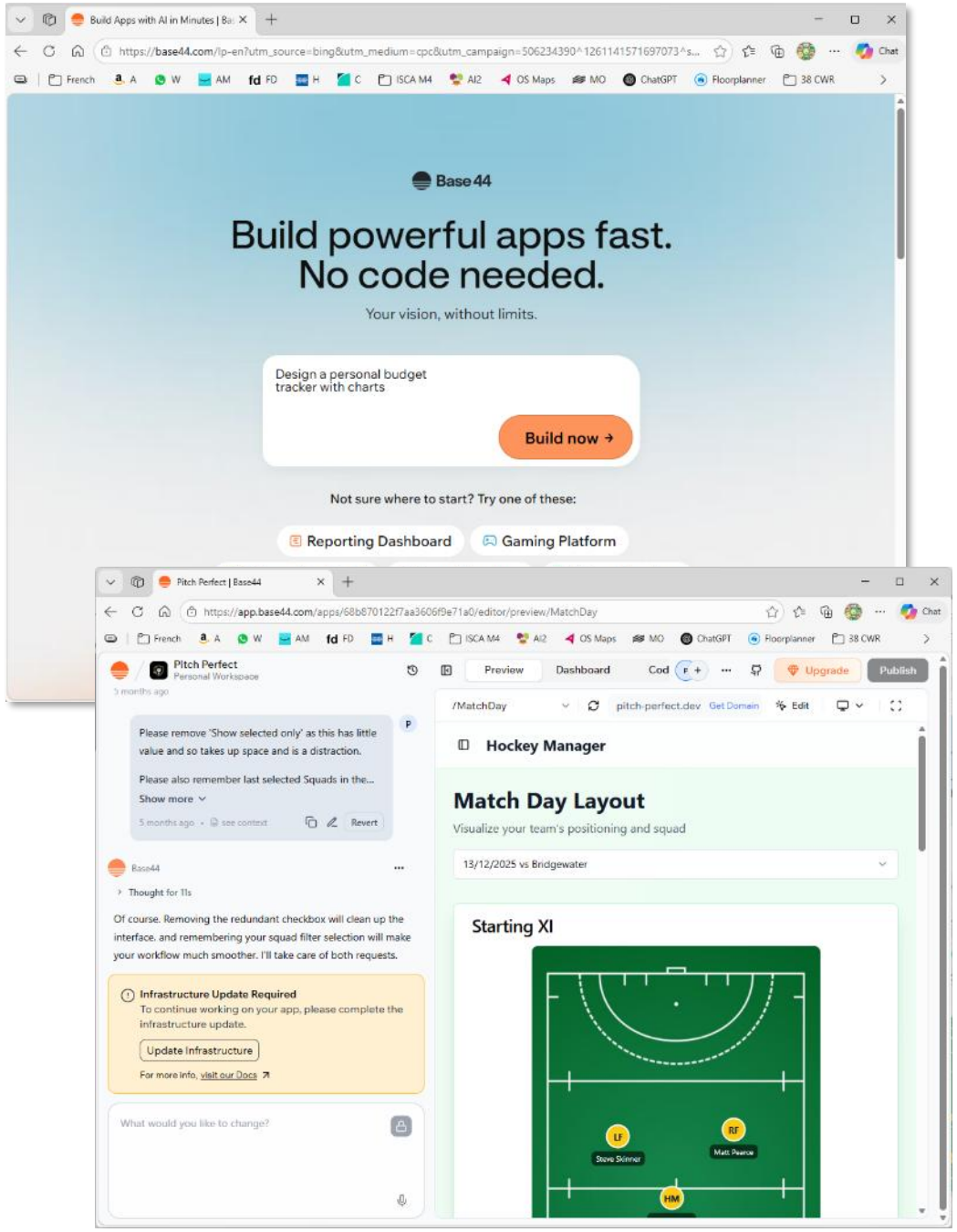
Base 44 is a good option you can try for free:

https://app.base44.com/

You can see my example app here: Pitch Perfect - a hockey team selection tool.

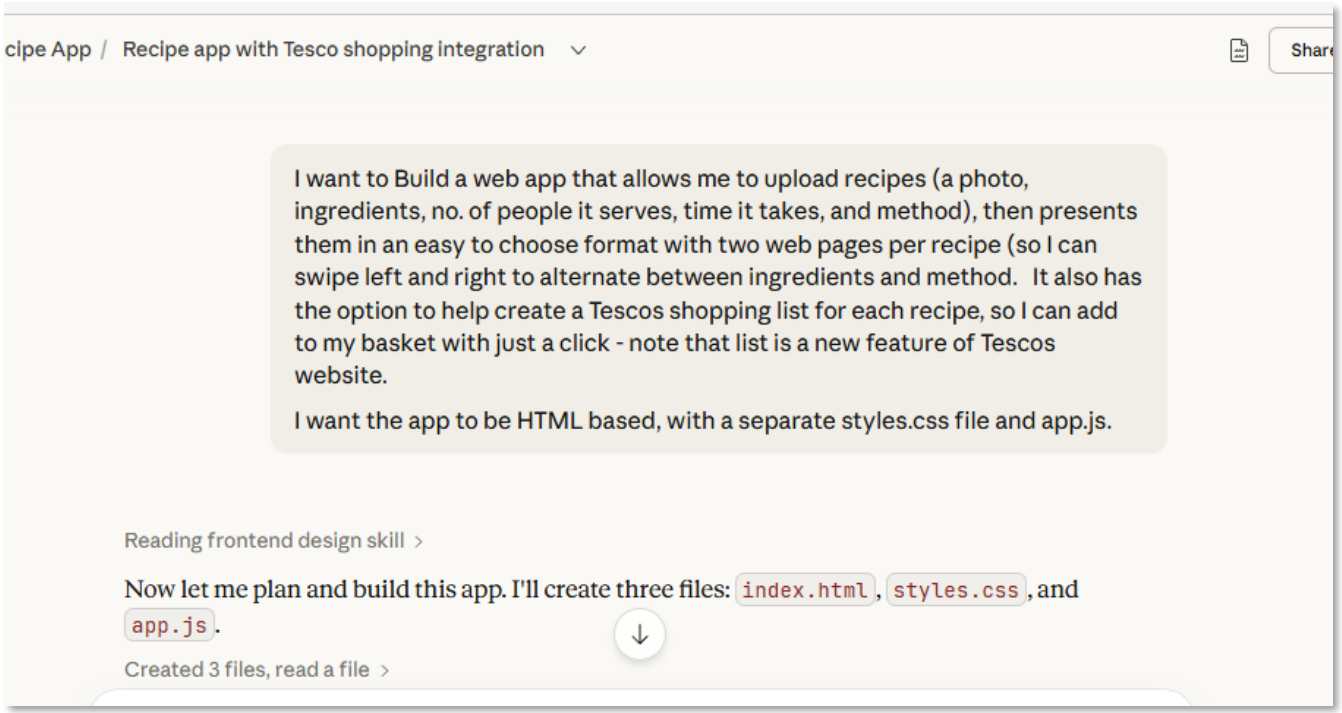
The free version is limited (just 40 prompts per month), but can be good to get started.

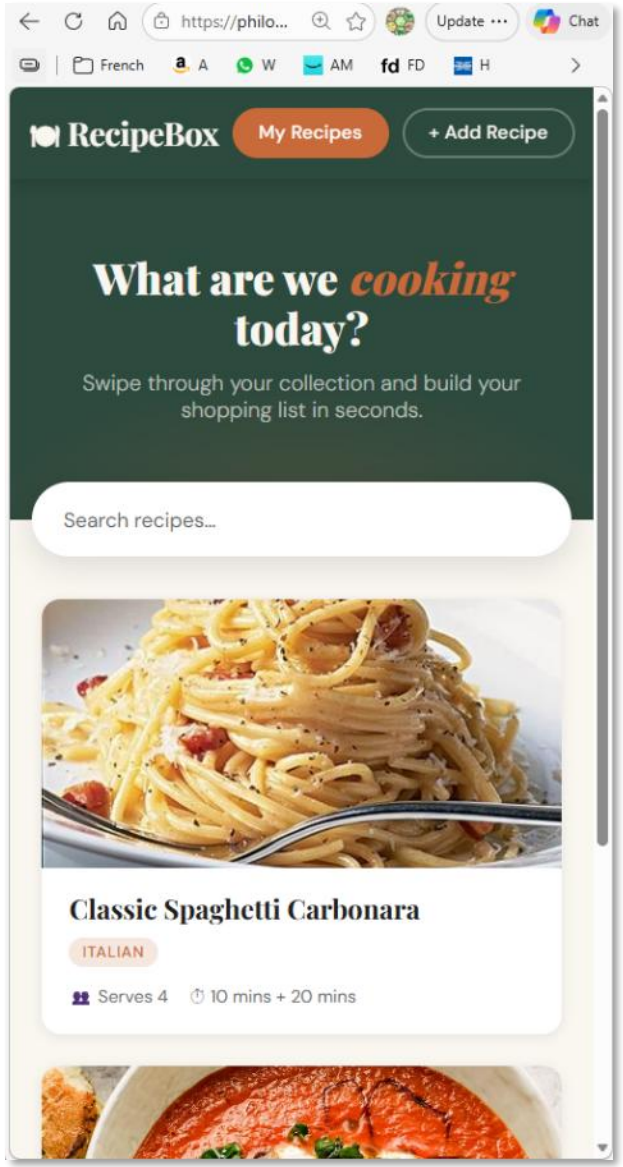
But I ran out of prompts, so started paying $20/month. But I wanted more control over the actual code, and realised I could do the same for free with chatbots, so that's what we'll do today…



VIBE-CODING A WEB APP

You can get started just by asking a chatbot to build a web page or web app. Here we're using claude.ai (click to sign-up on the free plan)…





https://philoutram.github.io/my-recipes/

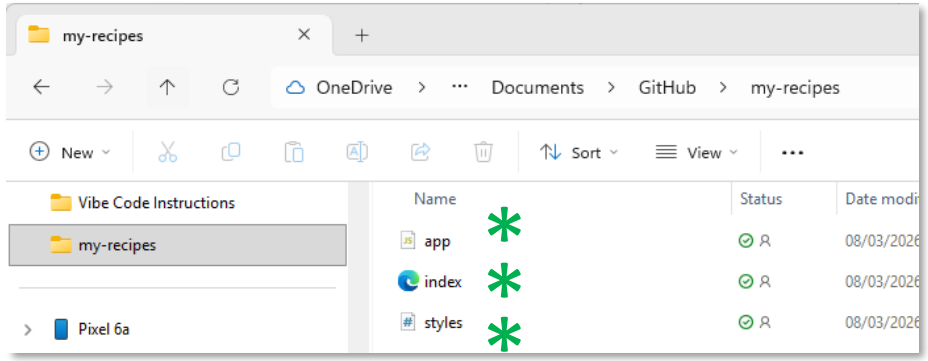
WEB APP COMPONENTS

Once built, web apps typically have 3 parts:

1. **.HTML File** – describes the basic layout and content of the web pages that make up your app, including all the text, links, etc.
2. **Styles** – the look and feel of the text, buttons, etc.
3. **JavaScript** – instructions for any buttons or actions that need to do things on your app.

Note: *when first starting, these three parts (html, styles and JavaScript) can all be stored in a single file - your* ***index.html*** *file.*

But as your app evolves, this single file would get large quickly and is harder for the LLM to iterate on (it has to regenerate the whole file every time it makes a change), so I prefer to split the 3 sections into 3 separate files (index.html, styles.css and app.js). You'd need to tell Claude to do this – see scaffold prompt on last slide.



Index · HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Apps</title>
  <style>
```

index.html contains text and layout

Styles · CSS

```
645 .serves-control { display: flex; align-items: center; gap: 14p
646 .serves-btn {
647   width: 30px;
648   height: 30px;
649   border-radius: 50%;
650   background: var(--green);
651   color: var(--white);
```

styles.css describes the sizes, colours, etc. of your web elements

App · JS

```
139
140 function startAddForm() {
141   editingId = null;
142   formTitle.textContent = 'Add a New Recipe';
143   clearForm();
144   resetIngredients();
145   resetSteps();
```

app.js is the JavaScript file where the (button) action happens!

WEB APP COMPONENTS – NEXT LEVEL

As your apps get more complicated, you might start considering other things like:

- **Data** – where do you need to go to get it? E.g. Petrol price data from a public government database (API)
- **Storage** – where are you going to store settings or app data (like recipes, or photos)?
- **Using AI** – do you want your web app to also use AI, e.g. to read the contents of web pages?
- **Map –** do you want to interact with maps or other cool features?

So many things you can do…but we don’t need to worry about these now

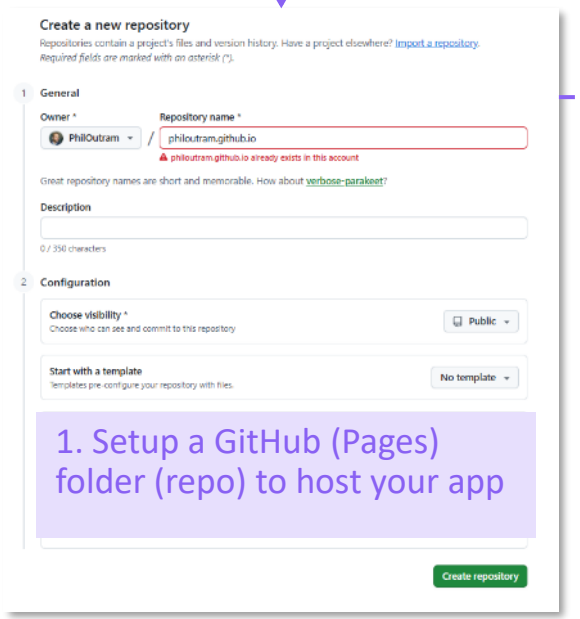
VIBING YOUR FIRST APP

What does it mean to vibe code your first app?

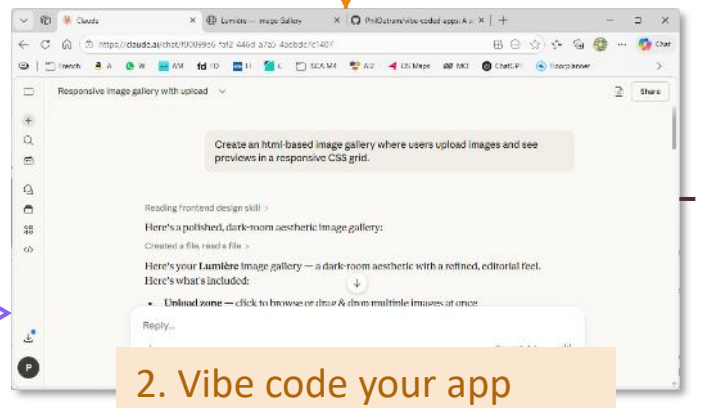
VIBE CODING FLOW

Sign up to Claude.ai

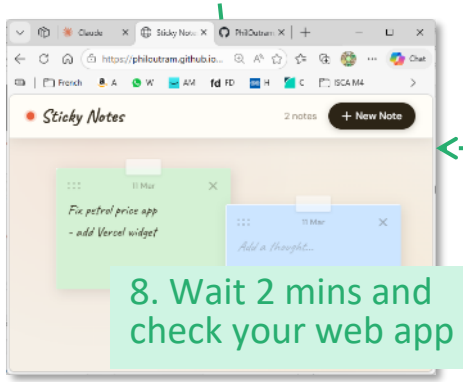
Sign up to GitHub



1. Setup a GitHub (Pages) folder (repo) to host your app

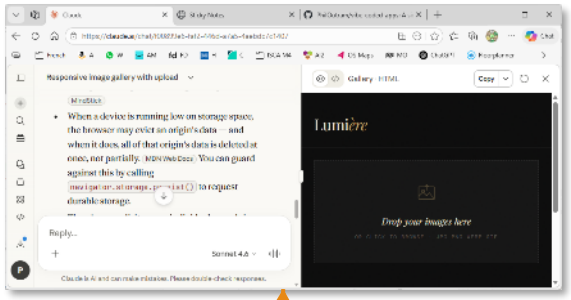


2. Vibe code your app



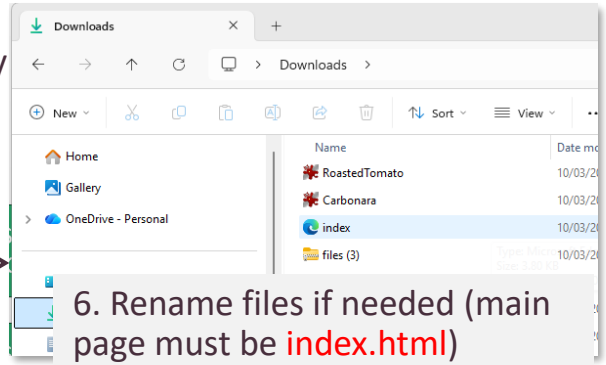
8. Wait 2 mins and check your web app

3 Use the Preview window to test your app.

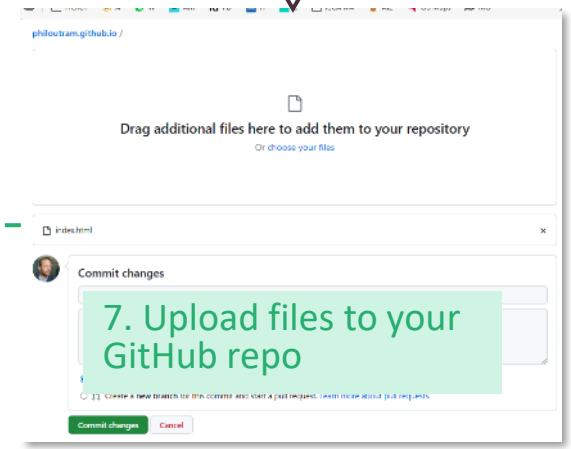


4. Keep chatting to iterate and improve your app

5. When ready to host, download the file(s)

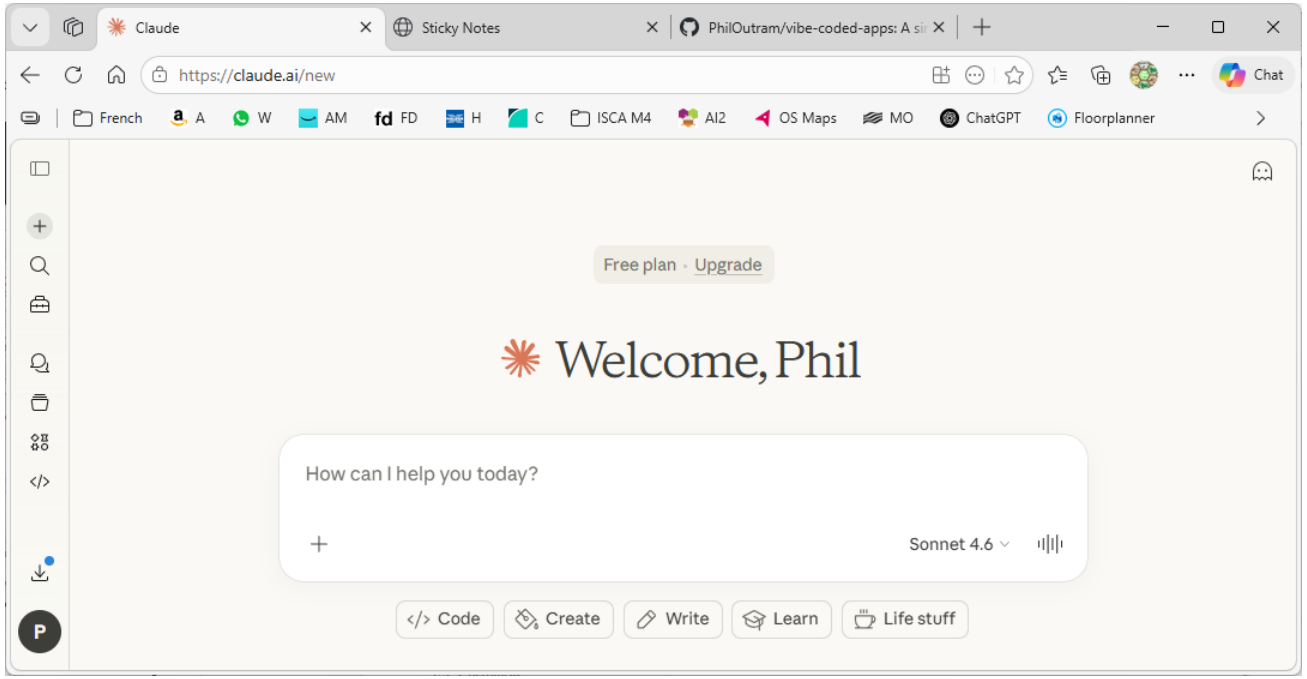


6. Rename files if needed (main page must be index.html)



7. Upload files to your GitHub repo

WHICH CHATBOT TO WRITE THE APP?



Gemini? Copilot? Claude? ChatGPT …would all work

Claude is popular (more so after their stance against the Pentagon). **Sign-up or log-in here: https://claude.ai/.**

As with DUNE/Copilot, you can refer back to prompts, or carry them on, at a later date.

APP 1: STICKY NOTES APP

Ideal for Beginners

Contains the basics with a web page, styles and uses javascript for some basic actions!

Copy-ready prompt:

```
Create a simple html-based sticky-notes web app with add/delete
buttons and save notes to localStorage. Make it clean and mobile-
friendly.
```

⚠️ WARNING: you may accidentally create a JSX app. This is a special type of app that is hosted by Claude and won't work outside Claude Chat. If that's the case, fix the prompt to ask to create an **html-based** web app.

Prompt extension:

```
Please add the ability to drag and drop to reposition sticky notes.
```

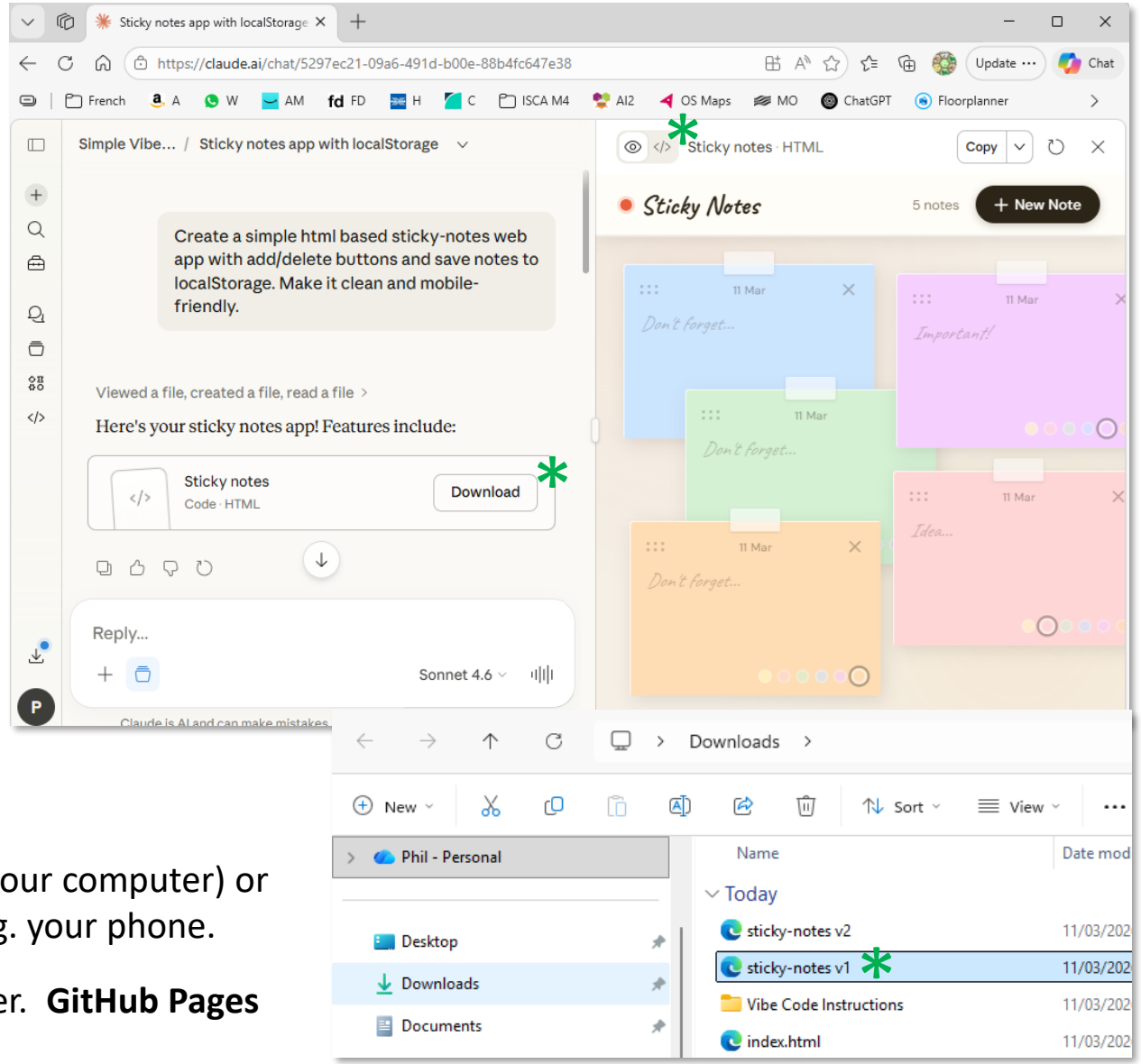
VIEWING YOUR APP

Preview
Claude very helpfully gives you a **preview window** on the right-hand side.

Viewing the code
If you're feeling brave, you can view the code it's generated by clicking on the **</>** button.

Download the file
Otherwise, you can download the html file that Claude has created.

Launch the app
Click on the **sticky-notes.html** file (or whatever Claude has called it) to launch the web page / app.



BUT…that doesn't help other people access it (if it's on your computer) or allow you to run it on devices other than your laptop, e.g. your phone.

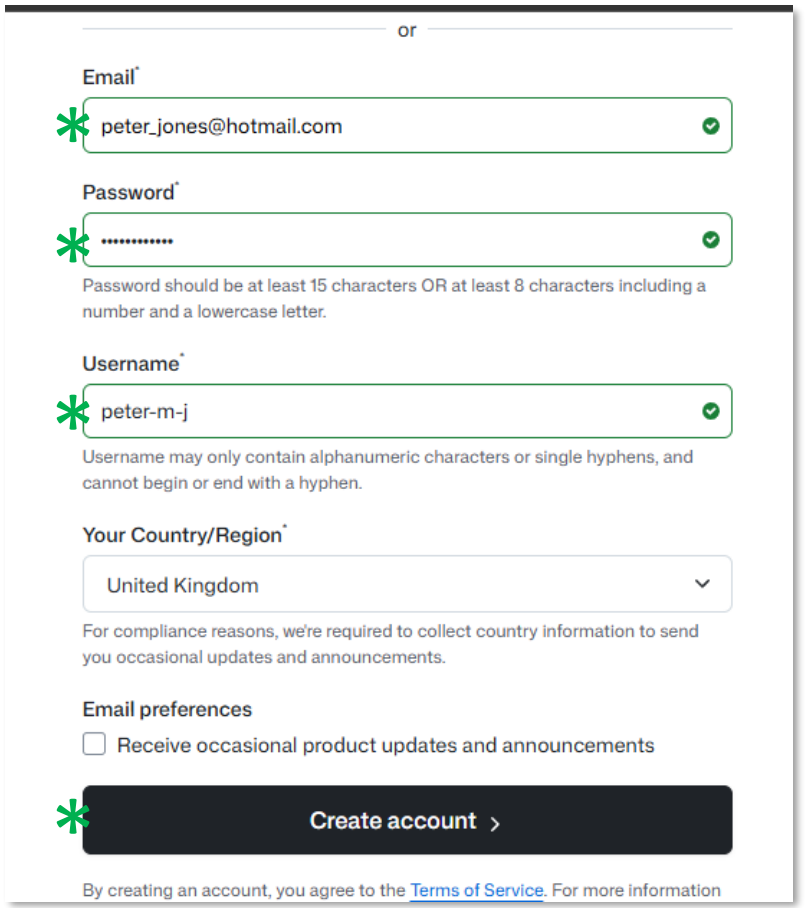
To do this, you need to **'host'** this file on a 3rd party server. **GitHub Pages** is a great free solution (let's set that up now!).

GITHUB SETUP

Do this once. You'll create a GitHub account, then set up a simple hub page at your own web address — a home base that will link to all the projects you build from here on.

STEP 1: SIGN UP FOR GITHUB ~3 MIN

1. Go to github.com — GitHub is a free website where you store and publish your web apps
2. Enter your **email address** and click **Sign up for Github**
3. Choose a password and a ***username*** — your username becomes part of your web address, so pick something you're happy to share publicly. **Make a note of it**, you'll need it in Step 2
4. If asked, complete the short visual puzzle to confirm you're not a robot
5. Click **Create account**
6. Check your email — GitHub will send a confirmation link. Click it to activate your account



COMPONENTS IN GITHUB PAGES

GitHub Pages Structure

In **GitHub Pages**, you will need to create the following folders (repositories):

1. **The root folder.**
   You won't store any apps here. You'll may just store a single web page with links to your apps as you create them.

2. **A sub-folder for each web app you're developing.**
   You can have as many web app projects and sub-folders as you like.

My root folder:
https://philoutram.github.io/

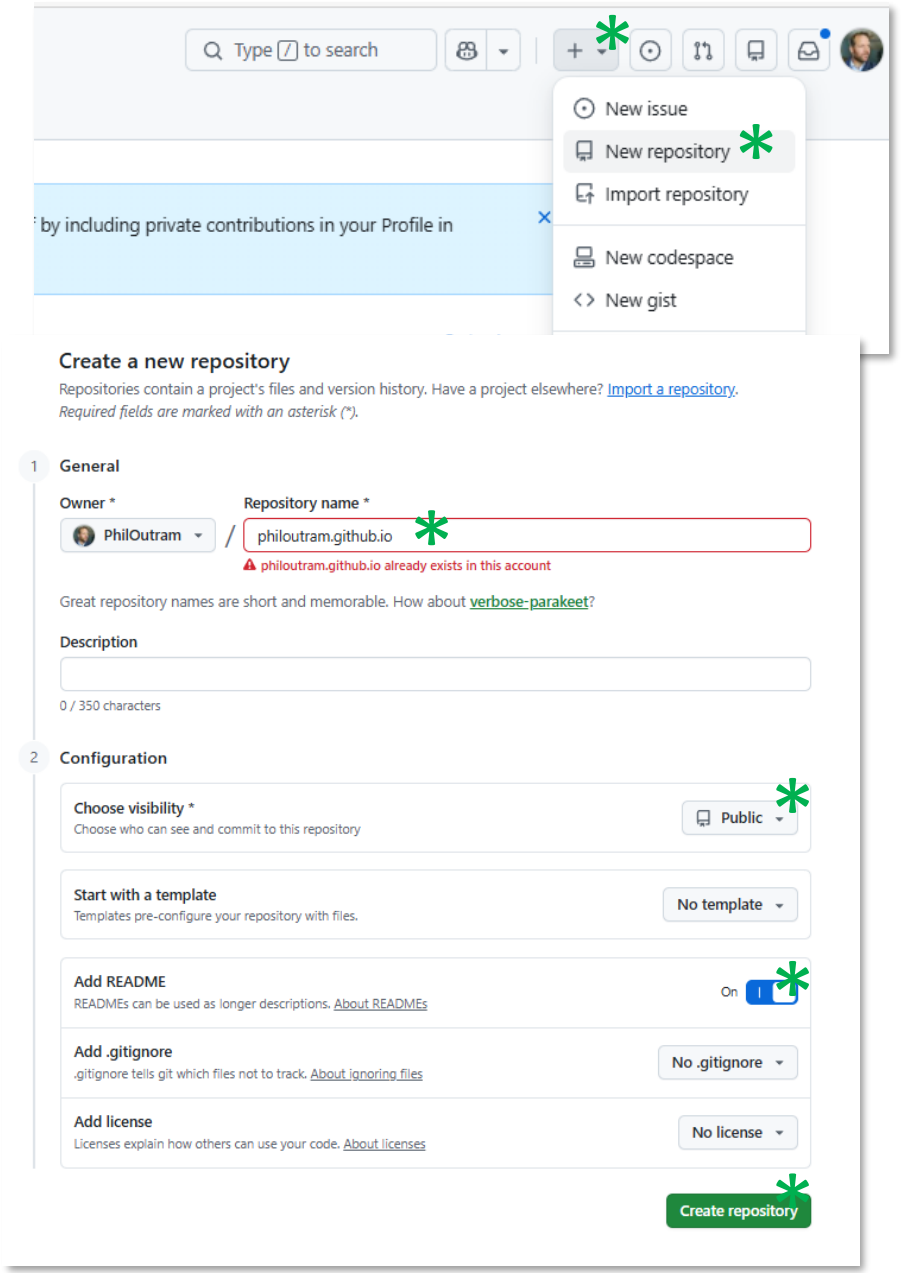
One of my project folders:
https://philoutram.github.io/my-recipes/

In GitHub parlance, a repository (or "repo") is a project folder on GitHub where your files live for a particular project

STEP 2: CREATE YOUR MAIN HUB REPOSITORY (FOLDER) ~2 MIN

1. Once logged in, click the + icon in the top right corner and select **New repository** — a repository (or "repo") is a project folder on GitHub where your files live
2. Name it ***yourusername.github.io*** — swap *yourusername* for your actual GitHub username

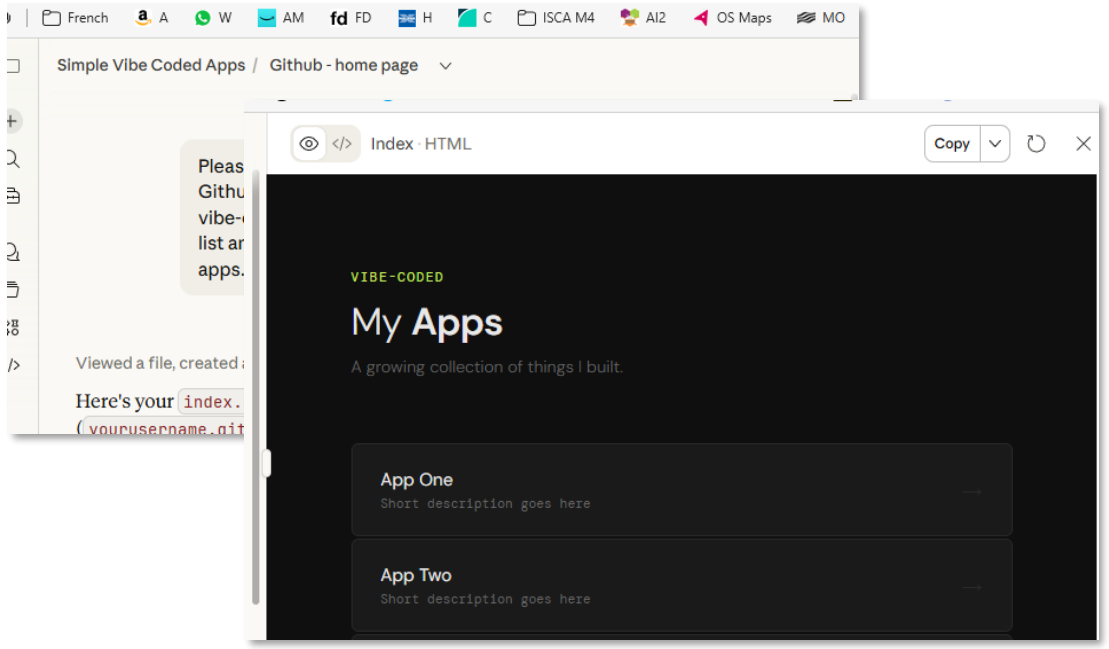
   ⚠️ The name must match your username exactly, including upper and lower case. This special name is what tells GitHub to turn this folder into your personal website.
3. Make sure visibility is set to **Public** — this means anyone can visit your site, which is what we want (you can't have a private site without a paid account)
4. Move the **Add README** file slider to On — this just creates a starter file, so the folder isn't empty
5. Click **Create repository**

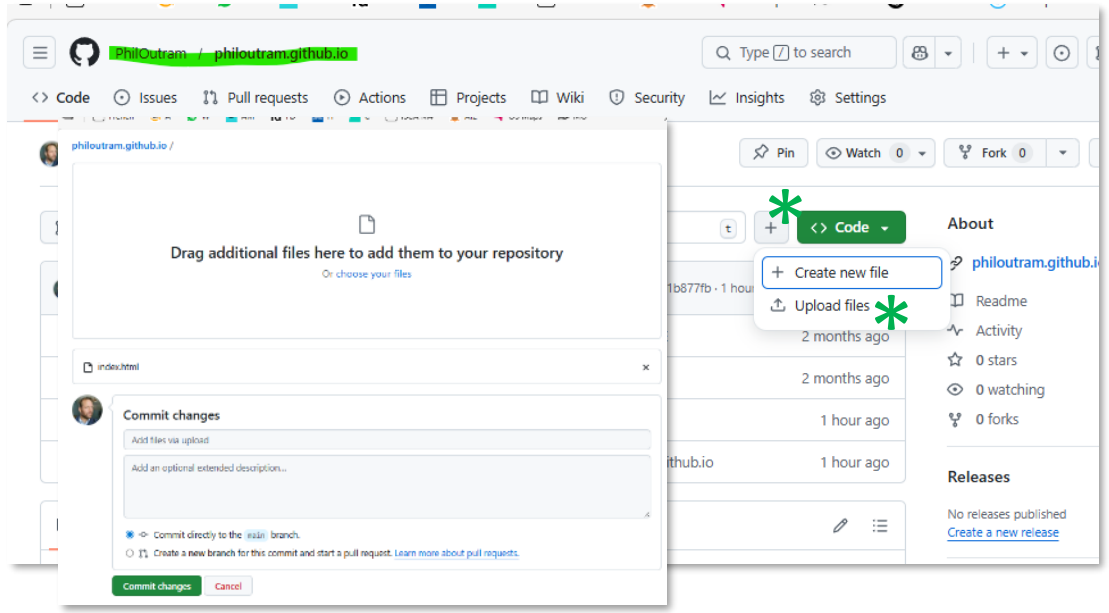


[OPTIONAL] STEP 3: ADD A SIMPLE HUB PAGE ~5 MIN

Rather than filling this main root repository with a specific project, you'll put a simple landing web page here — just your name and links to projects as you build them.

1. Ask Claude to create one for you (use prompt in notes)
2. Save the file Claude gives you as **index.html** on your computer.
3. In your repository on GitHub, click the **'+'** or **Add file** then **Upload files**. Then select your index.html.
4. Click Commit changes — "committing" just means saving the file to GitHub





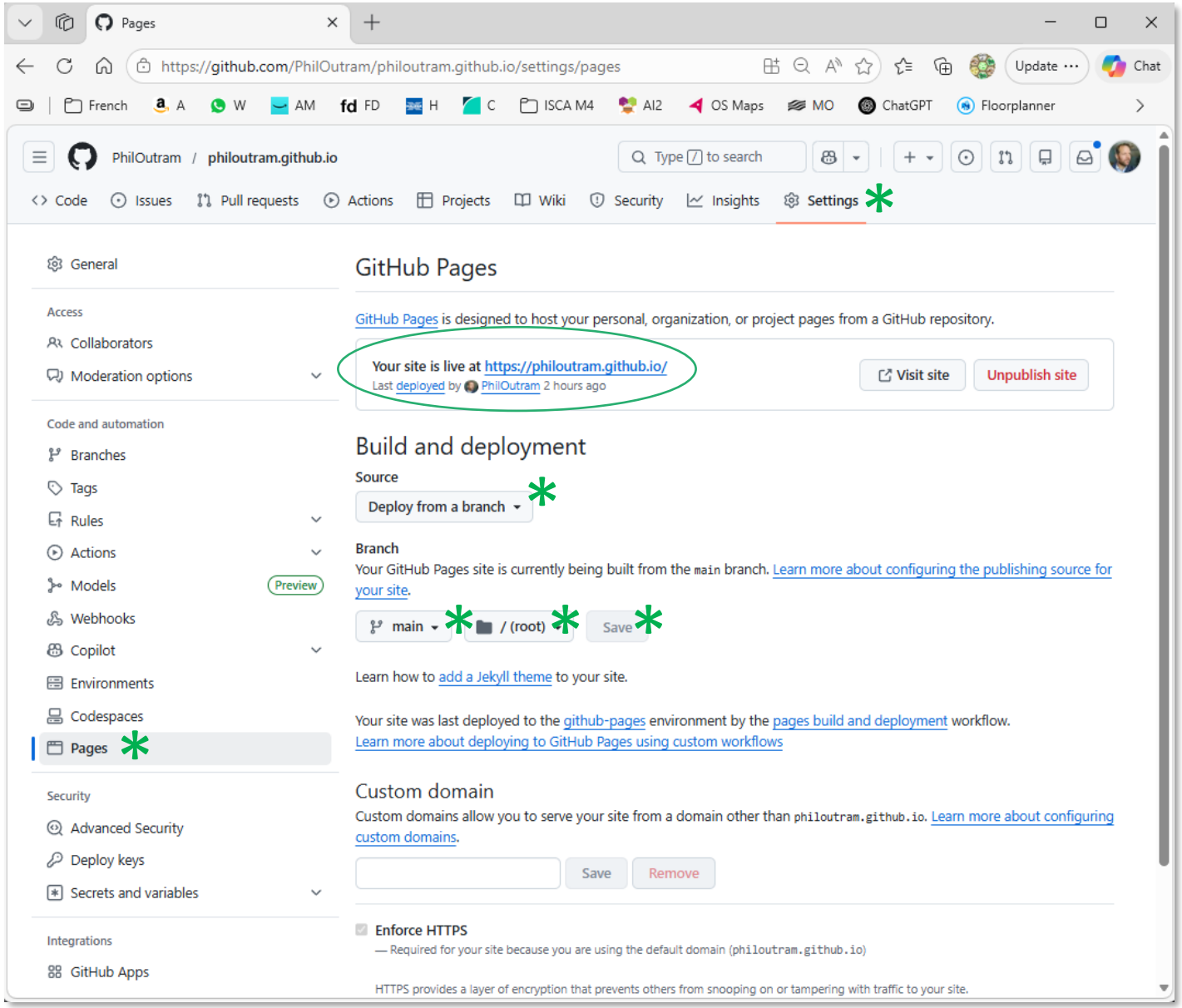
ACTIVATE YOUR LIVE SITE ~3 MIN

1. Click the **Settings** tab near the top of your repository page

2. In the left-hand menu, scroll down and click **Pages**

3. Under **Source**, select **Deploy from a branch**, set the branch to **main** and the folder to **/ (root)**, then click **Save**

💡 *"Branch" and "root" are technical terms — all they mean here is: use the **main** folder of your project, exactly as it is. If you already see a web address on this page, you're already set up.*



4. Your hub will be live at https://yourusername.github.io within a few minutes. Try refreshing if it doesn't load straight away

STARTING A NEW PROJECT

Do this each time you build a new web app. Every project lives in its own separate folder on GitHub and gets its own public web address — completely independent from your hub and from other projects.

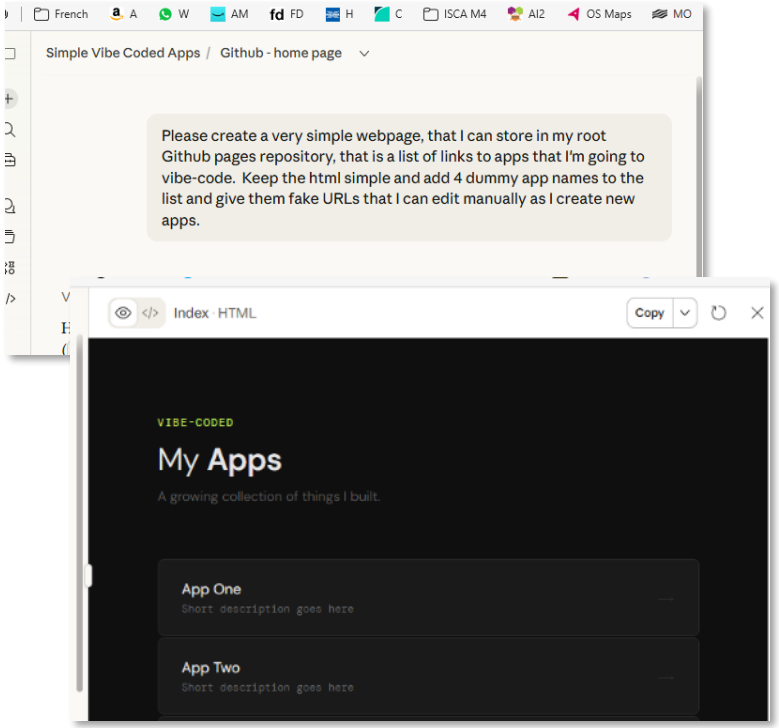
STEP 1: BUILD YOUR APP WITH CLAUDE ~VARIES

See following slides for further explanation on building apps

1. Open claude.ai and describe what you want to build. Be specific — the more detail you give, the better the result. For example: *"Build a single-page web app that lets me track my daily water intake. It should look clean and modern, work on mobile, and save my progress during the session."*

2. For simple apps, Claude will produce your app as a single file called index.html - this one file contains everything (the layout, the styling, and the logic). You can ask Claude to make changes by describing what you want to be different.

3. When you're happy with the result, copy the code and save it as **index.html** on your computer.

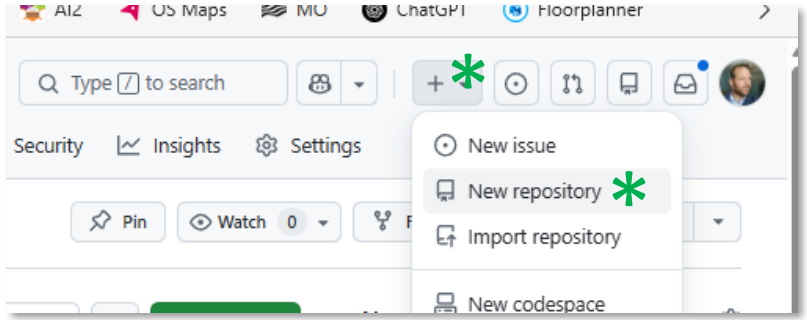


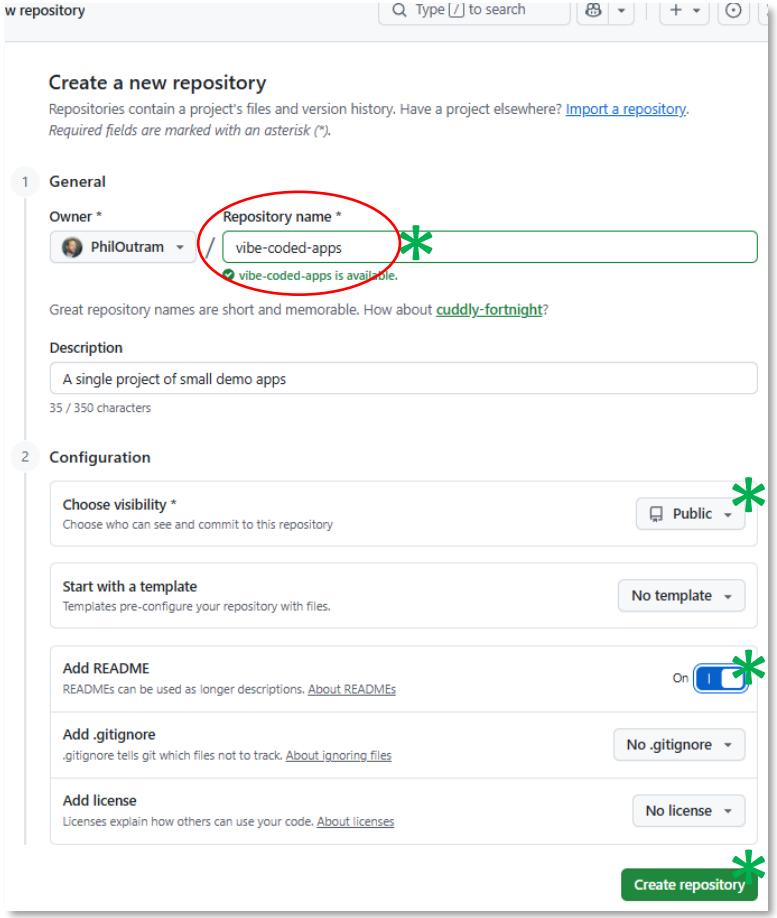
💡 **IMPORTANT:** Keep the filename as **index.html** — GitHub will serve this file automatically as the first page people see when they visit your project's web address.

STEP 2: CREATE A NEW REPOSITORY FOR YOUR PROJECT

~2 MIN

1. If not already, log in at github.com.
2. Click the **+** icon in the top right and select **New repository**.
3. Give it a **short, descriptive name** with no spaces — for example weather-app or quiz-game. **This name will appear in your project's web address** (e.g. **'vibe-coded-apps'** in this example)
4. Set **visibility** to **Public**.
5. Move the **Add README** file slider to On — this just creates a starter file, so the folder isn't empty
6. Click **Create repository**





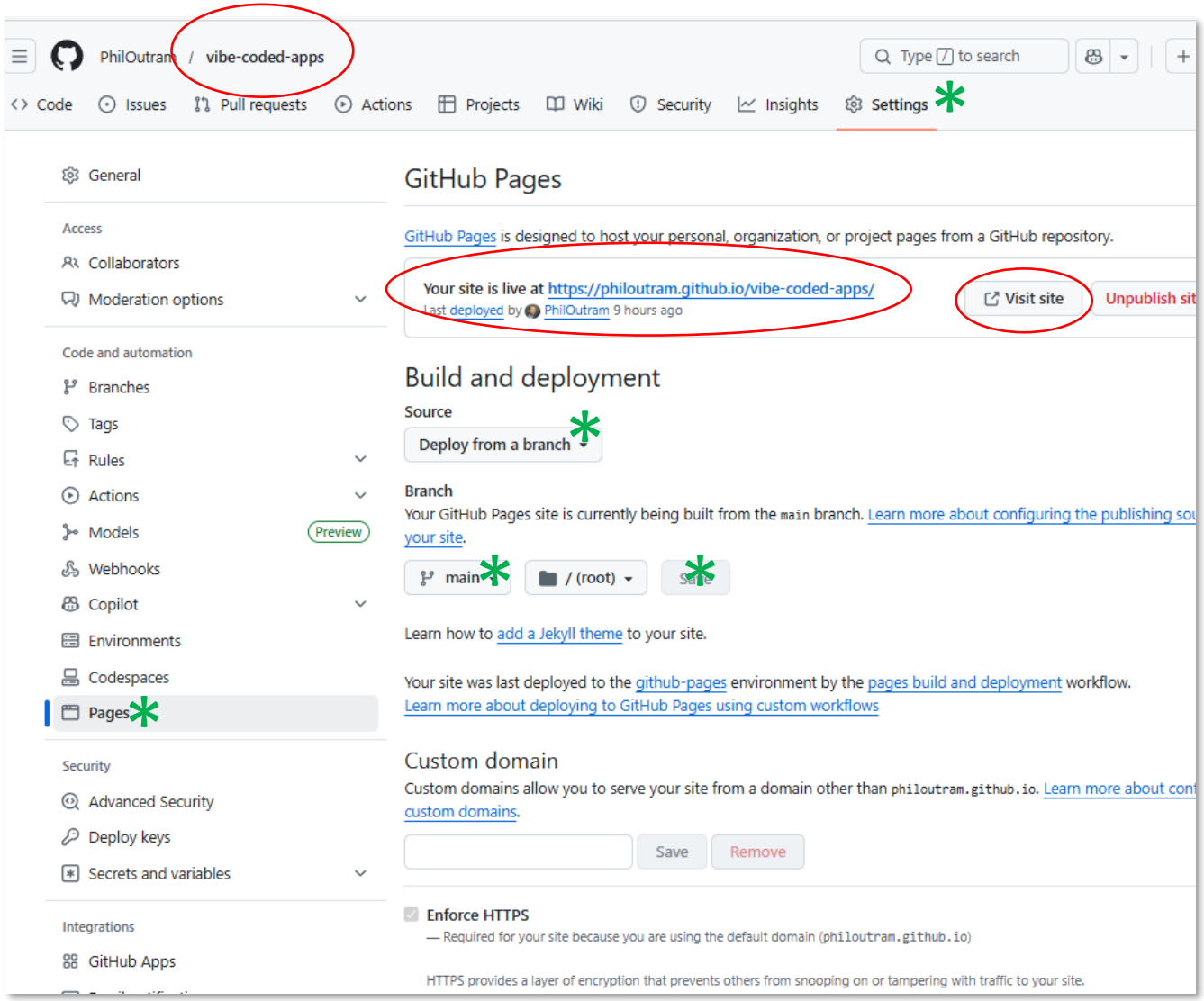
STEP 3: MAKE IT A WEB PAGE ~2 MIN

1. Click the **Settings** tab near the top of your repository page

2. In the left-hand menu, scroll down and click **Pages**

3. Under **Source**, select **Deploy from a branch**, set the branch to **main** and the folder to **/ (root)**, then click **Save**

💡 *"Branch" and "root" are technical terms — all they mean here is: use the main folder of your project, exactly as it is. If you already see a web address on this page, you're already set up.*



IMPORTANT: After two minutes or so, your new website will be **live** at https://yourusername.github.io/repositoryname. In this example, it's https://philoutram.github.io/vibe-coded-apps/. Try refreshing if it doesn't load straight away. NOTE that it obviously won't show your app yet, as you've not uploaded it – see the next step.

GitHub speak – Commit changes is to Save changes.

STEP 4: UPLOAD YOUR APP ~3 MINS

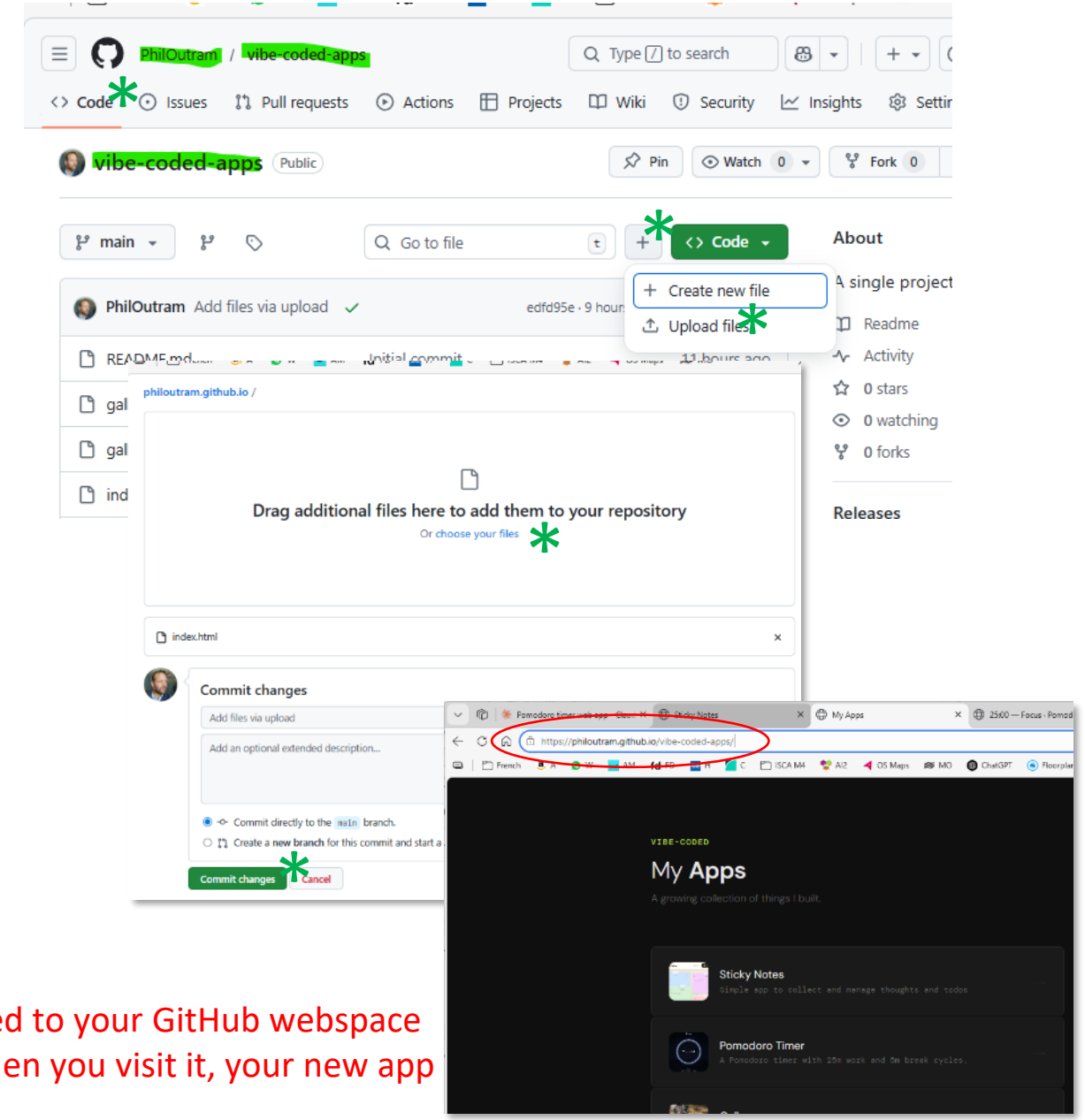
Once you've created your web app, are happy with the version in 'preview', have downloaded it and ensured the file is named index.html, you're readyto upload it to your site.

Upload your new web page:

1. Go to your new **repository** on GitHub. Make sure you're on the **<> Code** tab.

2. Click **'+'** then **Upload files** and choose or drag in your **index.html** file from your computer.

3. Click **Commit changes** to save the file



IMPORTANT:

After 2 minutes or so, your new **index.html** file will be pushed to your GitHub webspace (https://yourusername.github.io/repositoryname). Then when you visit it, your new app will be displayed.

BASIC WEB APP OVERVIEWS WITH PROMPTS

These have been tested

TO-DO APP

Application Logic and State

The app demonstrates managing arrays of objects by adding, updating, and removing tasks dynamically.

Filtering and Priority Handling

Users learn to implement filters to view tasks by priority, enhancing usability and focus.

Counters and Visual Feedback

Counters track the number of tasks, providing immediate feedback on the to-do list status.

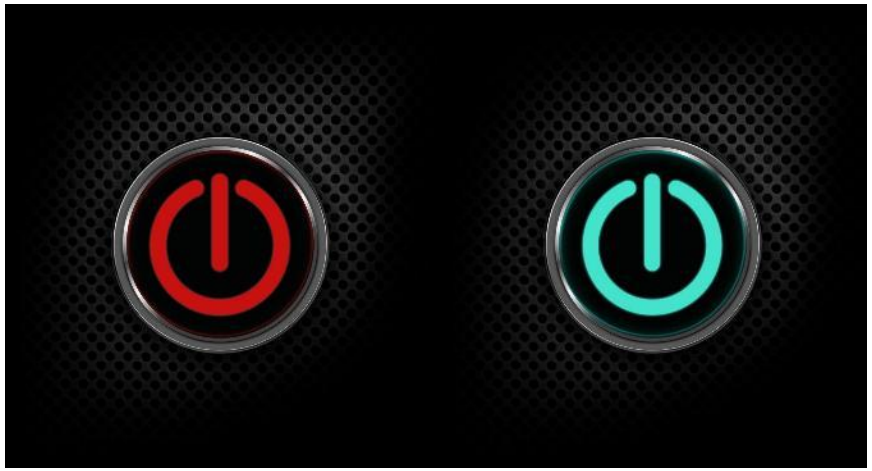
Separation of Concerns

Styling tasks with CSS reinforces the separation between logic and presentation layers.

Copy-ready prompt:

```
Build a to-do html web app with priority labels, filtering buttons, a
remaining-tasks counter, and localStorage persistence.
```

POMODORO TIMER



JavaScript Timing Functions

Learn to use setInterval and clearInterval to control timer updates efficiently.

State Management in Timer

Understand tracking states like active and paused to manage timer behavior effectively.

State Machine Concept

Reinforce building apps with multiple modes using the state machine mental model.

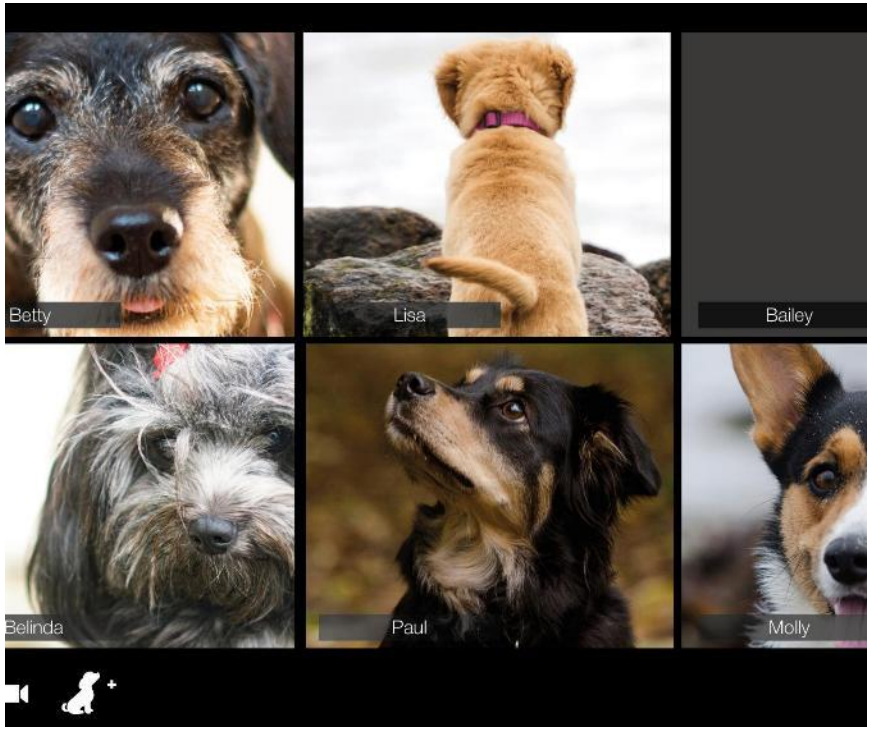
JavaScript DOM Interaction

See how JavaScript updates DOM elements repeatedly to reflect timer countdown.

Copy-ready prompt:

```
Create a Pomodoro timer with 25m work, 5m break, a circular
progress indicator, and start/pause/reset controls. Create
an html web app.
```

IMAGE GALLERY



User-Generated Content Handling

The app allows for file uploads and image previews using URL.createObjectURL, enabling dynamic content display.

Responsive Gallery Design

Thumbnails are organized in a CSS grid, creating a clean and responsive gallery layout adaptable to multiple devices.

Client-Side Processing

The browser interprets binary file data entirely client-side, teaching DOM manipulation and event handling effectively.

Usability and Creativity

The project emphasizes UI layout impact on usability and encourages creativity, building learners' confidence.

Copy-ready prompt:

```
Create an html-based image gallery where users upload
images and see previews in a responsive CSS grid.
```

NOTE: it actually needs more work than just this, but this will get you started… (you will need to think about persistent storage, albums, slideshow, etc.)

BASIC WEB APP OVERVIEWS WITH PROMPTS

These have NOT been tested

BUDGET CALCULATOR

Numeric Input and Validation

Users enter numeric data with validation ensuring accurate and reliable budget inputs.

Category Grouping and Aggregation

Expenses and incomes are grouped into categories with dynamic calculation of totals and subtotals.

Financial Summaries and Formatting

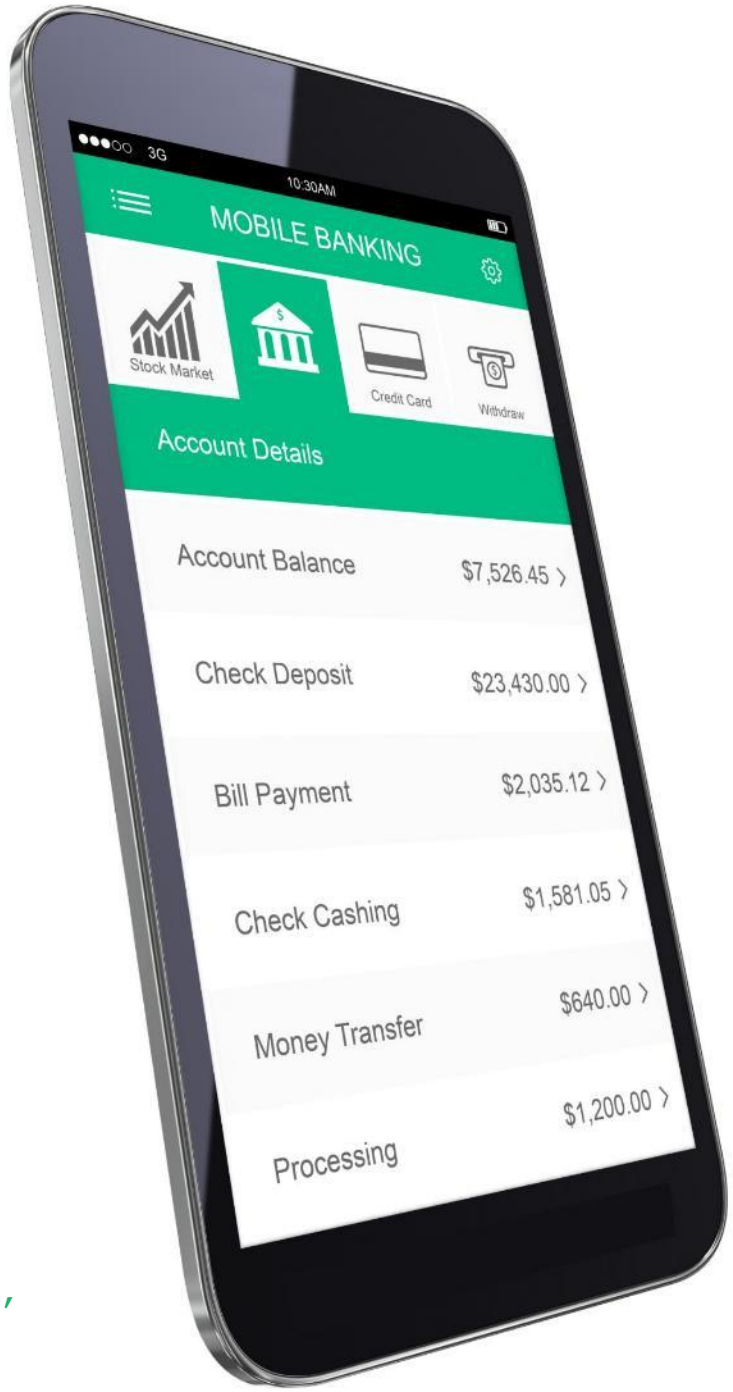
The app presents financial summaries using currency formatting for better readability and usability.

Usability and Extension Options

The calculator features clear inputs and supports extensions like exporting data or creating charts.

Copy-ready prompt:

```
Build a simple web-based (html) budget calculator with inputs for item,
category, and amount. Show total spend + category totals.
```



CSV VIEWER

File Handling in Browsers

Learners explore browser file handling using drag-and-drop and input elements without backend dependencies.

CSV Parsing and Display

CSV data is parsed and presented in a dynamic table to illustrate data and HTML structure relationships.

Error Handling Features

The app includes error handling to manage malformed CSV inputs and improve user understanding.

Empowering Client-Side JavaScript

Demonstrates powerful data workflows possible solely with client-side JavaScript, empowering new developers.

MORE FUN APPS

Ones that I want to build!

GARDEN CALENDAR

Another approach…adding the pre-step of asking for help to generate what is a slightly more complicated prompt.

This is the prompt I gave a generic chatbot, asking it to help come up with a prompt for Claude…

I want to vibe code an app to help me with my gardening.

I have a pack of 24 flower seeds. They have instructions on them. I'd like Claude to look at the 4 images that have the instructions for the 24 flowers and come up with an html based web-app that provides a calendar of when I should sew (indoors / outdoors) flowers, when they'll bloom, etc. I'd also love to do it with common vegetables and herbs, i.e. with bars on a calendar showing when to sew or plant and when to harvest.

The calendar should be the main page and be populated with a 'settings' sub-page where I can select or check all the flowers, herbs and vegetables that I want to include on my calendar. The calendar should then be populated accordingly.

I don't have the planting information for other plants, so Claude needs to get the information for common herbs and vegetables from the internet. But the settings page should also allow new plants to be added, including adding some key criteria (type: flower, herb, vegetable; sewing month indoors; sewing month outdoor; instructions for growing; harvest months; bloom months; etc). The information should be available in a pop-up when you click on the vegetable/herb/flower in the calendar. Also, if you click on the month, it would be very helpful to see general gardening tips for that month for healthy maintenance and enjoyment of your garden. Please suggest a good and detailed prompt that I can use to get Claude to vibe code an html based app with separate index.html, styles.css and app.js.

GARDEN CALENDAR

Suggested Claude Prompt:

I am building a gardening planner web app. I will provide four images that contain planting instructions for 24 flowers. Please read the images carefully and extract all planting data.
Using that data, plus general gardening information you find online for common vegetables and herbs, please build a complete web application with three files: index.html, style.css and script.js. Output the three files in clearly separated sections.
The app should work as follows:
Main functionality:
• The main page is a gardening calendar showing all 12 months.
• Each plant appears as coloured bars along the months for sowing indoors, sowing outdoors, blooming or harvesting.
• Clicking on any plant entry opens a pop-up with the plant's detailed information (type, instructions, sowing periods, bloom or harvest months).
• Clicking a month opens a pop-up with general gardening tips for that month (retrieved from reliable online sources).
Settings page: • Include a settings sub-page accessible from the main screen.
• It should list all flowers, vegetables and herbs with checkboxes. The calendar only shows the checked plants.
• The settings page must also allow adding new plants manually. For each new plant, let the user specify:
– name
– type (flower, herb or vegetable)
– sowing months indoors
– sowing months outdoors
– bloom months (if applicable)
– harvest months (if applicable)
– general growing instructions
• Newly added plants should appear in the calendar immediately.
Data handling: • Load the plant data for the 24 flowers from my images.
• Retrieve planting data for common herbs and vegetables from the internet.
• Store plant data in a simple JavaScript structure so the user can add more plants without a backend.
Technical requirements: • Produce fully functional code in three parts: index.html, style.css and script.js.
• Use semantic HTML, modern accessible CSS and clear vanilla JavaScript.
• No frameworks unless I ask for them later.
• The result should be copy-and-paste ready and work as a standalone web app.
Please begin by processing the images and then generate the full three-file scaffold.

FINAL THOUGHTS

PROMPTING & APP BUILDING TIPS

Scaffold Basic Structure

Create the initial project files like index.html, style.css, and script.js before adding features.

Specify Design Details

Clearly describe layout, colors, behavior, and accessibility for precise AI responses.

Iterate with Focused Updates

Request targeted code changes to maintain consistency and avoid full rewrites.

Fix for mobile

If you want to use it on your mobile, ask for a specific mobile layout and style. Check whether the layout is maximizing the use of your smaller screen and check it works in vertical mode. You can ask the LLM to change the layout, reducing the size given to less valuable items like the header.

Constrain to Simple Solutions

Set boundaries like avoiding frameworks to keep code simple and manageable.

Verify and Understand

Ask AI to explain output to catch bugs early and reinforce learning.

Error checking/handling

Use the Claude preview window to test your app. It might catch errors that you can copy and paste back into the chat and ask Claude to find the issue and fix it.

If something doesn't work as expected, explain this to the model and ask what it can do about it!

If your website isn't loading…have you definitely activated your repo on GitHub Pages? See GitHub instructions to switch on your site

Copy-ready prompt for scaffold:

```
Generate a clean
three-file scaffold:
index.html,
style.css, script.js,
using semantic HTML
and modern CSS.
```

SIMPLE LANDING PAGE IN ROOT REPO

Instead of adding an **index.html** to your main repo site (at yourusername.github.io), you can add an **index.md** mark-down file instead. This has a very simple feel and is very easy to edit.

1. You can upload the two files below to your main repo to get started.
2. Then click on the **index.md** file in your GitHub repo and click on the pencil symbol on the left to start editing it.
3. Take out my titles and links and replace with your own.
4. The format for a link is '- [My Recipes](my-recipes)'
   '- ' means add a bullet point
   '[My Recipes]' in square brackets is the text that is displayed
   '(my-recipes)' in curved brackets is the name of the repo you're trying to link to.

index.md _config.yml